

- **200 OK:** A requisição foi bem-sucedida e o servidor retornou os dados solicitados.
- **201 Created:** A requisição foi bem-sucedida e um **novo recurso foi criado** no servidor (comum em requisições POST).
- **400 Bad Request:** O servidor não entendeu a requisição devido a uma **sintaxe inválida** ou dados incorretos enviados pelo cliente.
- **401 Unauthorized:** A requisição exige autenticação. O usuário **não está autenticado**
- **403 Forbidden:** O usuário está autenticado, mas **não tem permissão** (autorização) para acessar o recurso solicitado.
- **404 Not Found:** O servidor **não encontrou o recurso** solicitado
- **500 Internal Server Error:** O servidor encontrou uma condição inesperada que o impediu de atender à requisição (um **erro genérico no código do servidor**).



V1__create-
table-
remedios.sql



V2__create-
table-
clientes.sql



V3__create-
table-
fornecedores.sql

```

@GetMapping("/listar")
@Operation(summary = "Listar todos os clientes", description = "Retorna a lista completa de clientes cadastrados")
@ApiResponses(value = {
    @ApiResponse(responseCode = "200", description = "Lista retornada com sucesso")
})
public List<Cliente> listar() {
    return this.service.listar();
}

@GetMapping("/{id}")
@Operation(summary = "Buscar cliente por ID", description = "Retorna o cliente correspondente ao ID fornecido")
@ApiResponses(value = {
    @ApiResponse(responseCode = "200", description = "Cliente encontrado",
        content = @Content(mediaType = "application/json",
            schema = @Schema(implementation = Cliente.class))),
    @ApiResponse(responseCode = "404", description = "Cliente não encontrado", content = @Content())
})
public Cliente cliente(
    @Parameter(description = "ID do cliente a ser buscado", required = true)
    @PathVariable Long id
) {
    return this.service.getCliente(id);
}

@PostMapping
@Operation(summary = "Criar um novo cliente", description = "Cadastra um novo cliente no sistema")
@ApiResponses(value = {
    @ApiResponse(responseCode = "201", description = "Cliente criado com sucesso",
        content = @Content(mediaType = "application/json",
            schema = @Schema(implementation = Cliente.class))),
    @ApiResponse(responseCode = "400", description = "Dados inválidos", content = @Content())
})
public void salvar(@RequestBody Cliente cliente) {
    this.service.salvar(cliente);
}

@PutMapping
@Operation(summary = "Atualizar um cliente", description = "Atualiza um cliente existente")
@ApiResponses(value = {
    @ApiResponse(responseCode = "200", description = "Cliente atualizado com sucesso",
        content = @Content(mediaType = "application/json",
            schema = @Schema(implementation = Cliente.class))),
    @ApiResponse(responseCode = "404", description = "Cliente não encontrado", content = @Content())
})

```

```

@RestController
@RequestMapping("/aluno")
public class AlunoController {

    private final AlunoService alunoService;

    public AlunoController(AlunoService alunoService) {
        this.alunoService = alunoService;
    }

    @GetMapping("/uuid/{uuid}")
    public ResponseEntity<Aluno> getAlunoByUuid(@PathVariable String uuid) {
        try {
            return ResponseEntity.ok(this.alunoService.getAlunoByUuid(uuid));
        } catch (NoSuchElementException e) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).body(null);
        }
    }

    @GetMapping("/get-all")
    public List<Aluno> getAlunos() {
        return this.alunoService.getAlunos();
    }

    @PostMapping
    public Aluno saveAluno(@RequestBody Aluno aluno) {
        return this.alunoService.saveAluno(aluno);
    }

    @PutMapping
    public ResponseEntity<Aluno> updateAlunoByUuid(@RequestBody Aluno aluno) {
        try {
            return ResponseEntity.ok(this.alunoService.updateAlunoByUuid(aluno));
        } catch (NoSuchElementException e) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).body(null);
        }
    }
}

```

```

@Schema(description = "Entidade que representa um aluno")
@Entity
@Table(name = "alunos")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@RequiredArgsConstructor

public class Aluno {

    @Schema(description = "Identificador do aluno")
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;

    @Schema(description = "Identificador único do aluno")
    @UuidGenerator
    private UUID uuid;

    @Schema(description = "Nome completo do aluno", example = "João da Silva")
    @NonNull
    private String nome;

    @Schema(description = "Matricula do aluno", example = "202500123")
    @NonNull
    private String matricula;

    @Schema(description = "E-mail do aluno", example = "joao@email.com")
    @NonNull
    private String email;

    @Schema(description = "Endereço do aluno")
    @Embedded
    @NonNull
    private Endereco endereco;

```

```

@NonNull
private Endereco endereco;

/* public Aluno(String nome, String email, String matricula, Endereco endereco)
{
    this.nome= nome;
    this.email = email;
    this.endereco = endereco;
    this.matricula = matricula;
}*/

public long getId() {
    return id;
}

public void setId(long id) {
    this.id = id;
}

public UUID getUuid() {
    return uuid;
}

public void setUuid(UUID uuid) {
    this.uuid = uuid;
}

public String getNome() {
    return nome;
}

public void setNome(String nome) {
    this.nome = nome;
}

public String getMatricula() {
    return matricula;
}

public void setMatricula(String matricula) {
    this.matricula = matricula;
}

public String getEmail() {

```

```
@Entity
@Table(name = "alunos")
@Getter
@Setter
@NoArgsConstructor
@RequiredArgsConstructor
public class Aluno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @UuidGenerator
    @Column(unique = true, updatable = false)
    @Schema(description = "UUID público do aluno", example = "edddd513-9ecd-45ac-8387-16256eae723a")
    private UUID uuid;

    @NonNull
    @Column(nullable = false)
    @Schema(description = "Nome do aluno", example = "Gabriel")
    private String nome;

    @NonNull
    @Column(nullable = false, unique = true)
    @Schema(description = "Matricula", example = "2026123")
    private String matricula;

    @NonNull
    @Column(nullable = false, unique = true)
    @Schema(description = "Email do aluno", example = "Gabriel@gmail.com")
    private String email;

    @NonNull
    @Embedded
    private Endereco endereco;
}
```

As camadas e suas responsabilidades

Uma requisição passa por três papéis bem definidos, cada um com uma única razão para mudar



Controller

CAMADA WEB

Recebe a requisição HTTP, delega para a Service e devolve status/resposta.



Service

CAMADA DE NEGÓCIO

Aplica regras de negócio, validações, defaults e orquestra transações.



Repository

CAMADA DE DADOS

Conversa com o banco de dados: persistência e consultas.

Regra: cada camada só conhece a que está logo abaixo dela. O Controller nunca deveria conhecer regra de negócio; a Service nunca deveria conhecer HTTP.